```
import numpy as np  #BMB2039
import pandas as pd  #PRAVEEN PK
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_error

# 1. Data Generation (Synthetic)
dates = pd.date_range('2018-01-01', periods=60, freq='M')
base_price = np.linspace(20, 100, len(dates))
seasonal = 5 * np.sin(np.linspace(0, 3*2*np.pi, len(dates)))
noise = np.random.normal(scale=2, size=len(dates))
prices = base_price + seasonal + noise
df = pd.DataFrame({
    'Past_Price': np.concatenate(([prices[0]], prices[:-1])),
    'Demand_Index': np.random.uniform(50, 100, len(dates)),
    'Supply_Index': np.random.uniform(40, 90, len(dates)),
    'Weather_Index': np.random.uniform(0.8, 1.2, len(dates)),
    'Price': prices
})

# 2. Feature-Target Split
X = df[['Past_Price', 'Demand_Index', 'Supply_Index', 'Weather_Index']]
y = df['Price']

# 3. Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, shuffle=True, random_state=0)

# 4. Model Training
model = RandomForestRegressor(n_estimators=100, random_state=0)
model.fit(X_train, y_train)

# 5. Prediction
y_pred = model.predict(X_test)

# 6. Evaluation Metrics
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
mae = mean_absolute_error(y_test, y_pred)
print(f'RMSE: {rmse:.2f}, MAE: {mae:.2f}')

# 7. Sample Predictions Table
pred_df = pd.DataFrame({'Actual': y_test.values, 'Predicted': y_pred})
print(pred_df.head())
```

```
/tmp/ipykernel_18027/1277442660.py:8: FutureWarning: 'M' is deprecated and will be removed in a future version, please use 'ME'
  dates = pd.date_range('2018-01-01', periods=60, freq='M')
RMSE: 5.78, MAE: 4.80
       Actual  Predicted
0   64.018223  59.627219
1   66.269776  61.622659
2  100.749283  90.402886
3   62.061644  64.428272
4   37.942487  33.107342
```